EVOLUTIONARY COMPUTATION — EXERCISE 5

Move Evaluations from Previous Iterations in Local Search

Steepest local search · two-edges exchange + inter-route exchange · list of improving moves

Instances A and B · 200 nodes each · 100 selected per tour
Report generated 2026-11-09

Wiktor Bocheński · Mikołaj Dybizbański

Code: https://github.com/mikolajdybizbanski/EC/blob/main/Lab4/main.cpp

1. Description of the problem

We are given two problem instances, **A** and **B**, each listing 200 nodes. Every node is described by three integers: its `x` and `y` coordinates on a plane, and a `cost`. Node indices are 0-based, taken from the row order in the instance file.

Goal. Select exactly 50% of the nodes (rounded up if the count is odd — for 200 nodes, exactly 100) and arrange the selected nodes into a **Hamiltonian cycle**: a closed path that visits every selected node exactly once and returns to its start. The objective to minimize is:

```
objective = total length of the cycle + sum of costs of the selected nodes
```

Distances between nodes are **Euclidean**, computed from the `(x, y)` coordinates and **rounded mathematically** (round-half-away-from-zero) to the nearest integer. The distance matrix is built once, immediately after reading an instance; from that point on, every optimization method only ever reads the distance matrix and the cost list — never the raw coordinates — so the same methods would work unchanged for an instance given purely as a distance matrix, with no coordinates at all.

A solution is a sequence of 100 distinct node indices, in visiting order; the cycle is implied by wrapping the last node back to the first. A solution is valid iff it has exactly `ceil(n/2)` nodes, every index is in range, and no index repeats.

2. Implemented methods

Exercise 3 showed that the best neighborhood is **two-edges exchange** (intra-route) together with the **exchange of a selected and an unselected node** (inter-route). This exercise speeds up the **steepest** local search on that neighborhood by reusing move evaluations from previous iterations. Both methods start from the **same 200 random solutions**.

1. **Steepest LS, edges exchange (baseline)** — the exercise 3 implementation: in every iteration the delta of every move in the neighborhood is computed (≈15 000 moves), and the best improving one is applied.
2. **Steepest LS, edges exchange, list of improving moves (LM)** — improving moves are kept in a list ordered from the best to the worst delta and reused in later iterations.

How moves are stored

A move is stored by the **nodes** that define it, not by tour positions. Then its delta stays exact for as long as those nodes keep the same neighbors, and the move can be checked against the current solution at any time:

- **Two-edges exchange** `(a, a', b, b')`: remove edges (a,a') and (b,b'), add (a,b) and (a',b'). Delta = d(a,b) + d(a',b') – d(a,a') – d(b,b').
- **Inter-route exchange** `(p, v, n, w)`: remove selected node v lying between p and n, insert unselected node w in its place. Delta = d(p,w) + d(w,n) – d(p,v) – d(v,n) + c(w) – c(v).

Browsing the list — the three situations

LM is browsed from the best move. For a two-edges exchange we look at both removed edges in the current tour:

- **At least one edge no longer exists** → the move can never be applied again: remove it from LM.
- **Both edges exist, but with different relative directions** (a→a' and b'→b) → applying the reconnection now would split the tour into two cycles. Leave the move in LM (a later segment reversal can make it applicable again) and browse further.
- **Both edges exist with the same relative direction** (a→a' and b→b', or both reversed: a'→a and b'→b) → apply the move and remove it from LM.

An inter-route exchange is applicable when w is still unselected and v still sits between p and n (in either order; both of v's edges are removed, so their direction does not matter). Otherwise it is removed from LM.

Which moves are evaluated

- **Initial solution**: all moves are evaluated. For every pair of edges a→a', b→b' we add *both* the currently applicable reconnection (a,b)+(a',b') and the one with **relatively inverted edges** (a,b')+(a',b), stored as the move on edges (a,a') and (b',b).
- **After a two-edges exchange**: every two-edges exchange using one of the two new edges (both reconnections), and every inter-route exchange of the four affected nodes.
- **After an inter-route exchange** (v → w between p and n): every two-edges exchange using the new edges (p,w) and (w,n), the inter-route exchanges of p, w and n, and the exchange of the newly unselected v with every selected node.

A move's delta depends only on its own nodes, so a move that was not improving when evaluated is never improving later. A reversal changes edge directions but not deltas, and the inverted reconnections already in LM cover that. So LM always contains every improving move, and the best applicable move in LM is exactly the move the baseline would choose (up to ties).

Stopping rule. The suggested algorithm repeats *until LM is empty*. With inverted moves kept in LM, the list may never empty: a move whose edges keep opposite directions stays there for good. Our first version looped forever on this. Instead, when no stored move is applicable, all moves are evaluated once more. If still no move is applicable right after that full evaluation, the solution is a local optimum and the search stops.

3. Pseudocode

3.1 Steepest local search with a list of improving moves

```
listLocalSearch(x):
    LM = empty set ordered by delta (best first)
    evaluateAll(x)                      # add improving moves, incl. inverted ones
    justEvaluatedAll = true
    repeat:
        m = first applicable move in LM  # browse, see 3.2
        if m found:
            x = m(x);  remove m from LM
            evaluate moves touching the new edges / changed nodes (see section 2)
            justEvaluatedAll = false
        elif justEvaluatedAll:
            return x                    # local optimum
        else:
            evaluateAll(x);  justEvaluatedAll = true
```

3.2 Browsing LM

```
firstApplicable(LM, x):
    for m in LM from the best:
        if m is a two-edges exchange (a, a', b, b'):
            dir1 = direction of edge (a, a') in x     # +1 a->a', -1 a'->a, 0 missing
            dir2 = direction of edge (b, b') in x
            if dir1 == 0 or dir2 == 0: remove m from LM          # edge gone
            elif dir1 != dir2:        continue                    # keep for later
            else:                     return m                    # same direction
        else:  # inter-route exchange (p, v, n, w)
            if w unselected and v is between p and n: return m
            else: remove m from LM
    return none
```

3.3 Evaluating a pair of edges (both reconnections)

```
evaluateEdgePair(a -> a', b -> b'):             # edges in current tour direction
    if the edges share a node: return
    removed = d[a][a'] + d[b][b']
    add (a, a', b,  b') with delta d[a][b]  + d[a'][b'] - removed    # applicable now
    add (a, a', b', b ) with delta d[a][b'] + d[a'][b]  - removed    # inverted
    (a move is added to LM only if its delta < 0)

apply two-edges exchange (a, a', b, b') with direction dir:
    i = pos[a]  if dir == +1 else pos[a']     # start of the first removed edge
    j = pos[b]  if dir == +1 else pos[b']     # start of the second removed edge
    reverse tour[min(i,j)+1 .. max(i,j)]     # adds (a,b) and (a',b')
```

4. Computational experiment

Each method: 200 runs per instance. Both methods of this exercise start from the same 200 random solutions. Cells show **average (min – max)**; the best average per instance is shown in **green**.

Objective function

Method	Instance A	Instance B
This exercise (same 200 random starting solutions)		
Steepest LS, edges exchange (baseline)	76180.4 (73561 – 79451)	**52491.1 (50248 – 55528)**
Steepest LS, edges exchange, list of improving moves	76168.1 (73561 – 79451)	52497.9 (50230 – 55306)
Previous methods (exercises 1–3)		
Random solution	262519.4 (238710 – 294111)	207338.6 (180318 – 239655)
Nearest neighbor (end only)	87476.6 (84362 – 90075)	59397.3 (56007 – 63166)
Nearest neighbor (any position)	74347.0 (72328 – 77022)	59480.3 (52915 – 66164)
Greedy cycle	73797.4 (72590 – 75281)	59709.5 (53016 – 63736)
2-regret (NN, any position)	122860.7 (111527 – 133181)	74763.4 (69032 – 80767)
Weighted 2-regret + delta (NN, any position)	75561.2 (72419 – 78504)	55698.3 (50485 – 61406)
2-regret (greedy cycle)	121007.2 (112654 – 130924)	73403.4 (69455 – 78142)
Weighted 2-regret + delta (greedy cycle)	73348.7 (72011 – 75532)	57895.5 (52731 – 63370)
Steepest LS, nodes exchange, random start	90051.5 (83416 – 97595)	66084.1 (58864 – 75837)
Steepest LS, nodes exchange, greedy start	73017.1 (71955 – 74682)	54292.1 (49978 – 61265)
Steepest LS, edges exchange, random start	76245.8 (73704 – 80426)	52597.5 (50149 – 55884)
Steepest LS, edges exchange, greedy start	72996.8 (71955 – 74682)	53091.3 (49346 – 60354)
Greedy LS, nodes exchange, random start	87787.9 (81532 – 98399)	63784.8 (58120 – 72965)
Greedy LS, nodes exchange, greedy start	73016.7 (71955 – 74682)	54298.5 (49913 – 61265)
Greedy LS, edges exchange, random start	76273.1 (74090 – 78637)	52935.7 (50579 – 55914)
Greedy LS, edges exchange, greedy start	**72979.8 (71955 – 74682)**	53094.7 (49727 – 60266)

Previous rows are taken from 3_local/results. The baseline here equals exercise 3's steepest LS, edges exchange, random start; it differs slightly only because it uses another set of 200 random starting solutions.

Running time [ms per run]

Method	Instance A	Instance B
This exercise (search only)		
Steepest LS, edges exchange (baseline)	7.558 (6.630 – 8.887)	7.517 (6.519 – 8.775)

Method	Instance A	Instance B
Steepest LS, edges exchange, list of improving moves	**3.838 (3.169 – 4.655)**	**3.728 (3.192 – 4.529)**
Previous methods (exercises 1–3, including construction of the start)		
Random solution	0.006 (0.004 – 0.009)	0.005 (0.004 – 0.006)
Nearest neighbor (end only)	0.037 (0.032 – 0.043)	0.034 (0.032 – 0.050)
Nearest neighbor (any position)	0.886 (0.873 – 0.917)	0.861 (0.847 – 0.890)
Greedy cycle	0.962 (0.951 – 0.999)	0.930 (0.911 – 0.962)
2-regret (NN, any position)	1.324 (1.301 – 1.356)	1.332 (1.312 – 1.365)
Weighted 2-regret + delta (NN, any position)	1.317 (1.298 – 1.355)	1.341 (1.319 – 1.515)
2-regret (greedy cycle)	1.541 (1.496 – 1.815)	1.545 (1.525 – 1.611)
Weighted 2-regret + delta (greedy cycle)	1.718 (1.492 – 3.830)	1.547 (1.523 – 1.611)
Steepest LS, nodes exchange, random start	10.625 (8.395 – 16.335)	10.652 (7.682 – 16.370)
Steepest LS, nodes exchange, greedy start	1.853 (1.562 – 2.569)	2.152 (1.420 – 3.014)
Steepest LS, edges exchange, random start	7.617 (6.476 – 10.921)	7.497 (6.236 – 8.585)
Steepest LS, edges exchange, greedy start	1.774 (1.558 – 2.881)	2.189 (1.533 – 3.053)
Greedy LS, nodes exchange, random start	5.317 (2.794 – 9.953)	4.967 (2.625 – 10.444)
Greedy LS, nodes exchange, greedy start	2.860 (2.103 – 4.351)	3.429 (1.980 – 7.962)
Greedy LS, edges exchange, random start	3.557 (2.042 – 6.779)	3.462 (1.971 – 5.080)
Greedy LS, edges exchange, greedy start	2.913 (2.132 – 5.730)	3.752 (2.294 – 6.909)

Times of this exercise cover the local search only (the random starts are generated beforehand). Building a random start takes ≈0.005 ms, so the comparison with exercise 3 is fair. Measured on Apple Silicon, clang++ -O2, single thread.

Search effort (average per run)

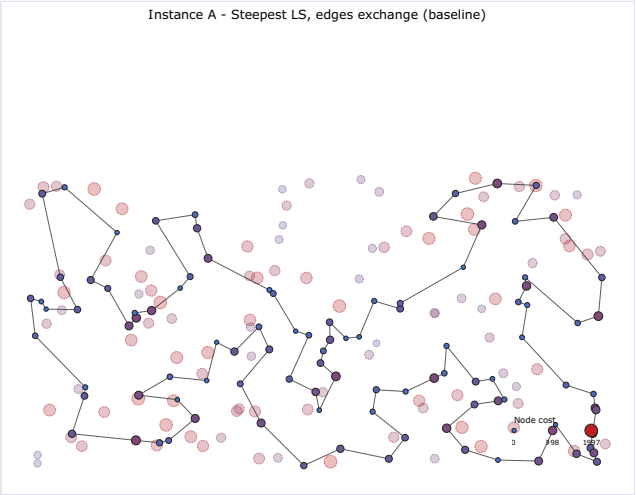
Inst.	Method	Evaluated moves	Applied moves	Full evaluations	Peak LM size
A	Steepest LS, edges exchange (baseline)	1,990,642	133.1	every iteration	—
A	Steepest LS, edges exchange, list of improving moves	144,127	133.1	2.0	11,411
B	Steepest LS, edges exchange (baseline)	1,990,494	133.0	every iteration	—
B	Steepest LS, edges exchange, list of improving moves	144,023	133.0	2.0	11,009

Raw data: 5_deltas/results/A_summary.csv, B_summary.csv.

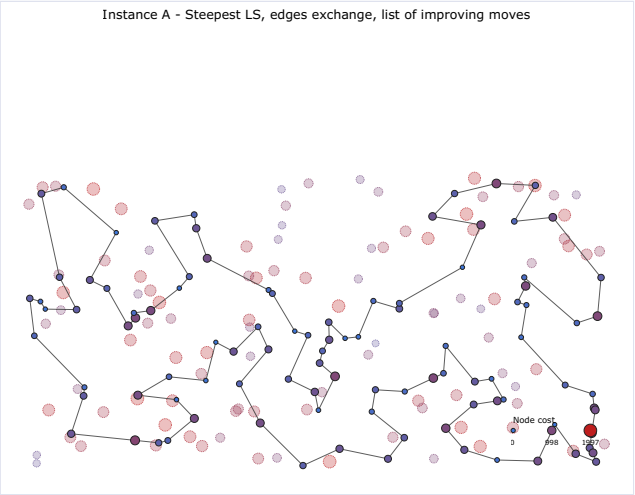
5. 2D visualizations of the best solution per instance/method

Node color and size encode cost (small/blue = cheap, large/red = expensive). Unselected nodes use the same cost encoding but are drawn translucent with a dashed outline; the closed tour is drawn as a dark path.

Instance A

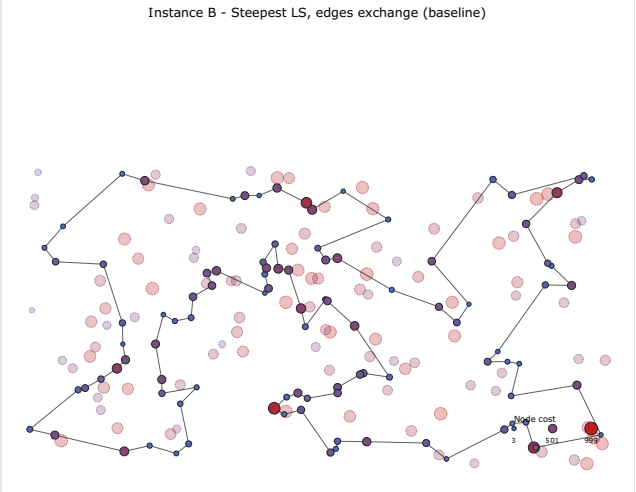


Steepest LS, edges exchange (baseline)

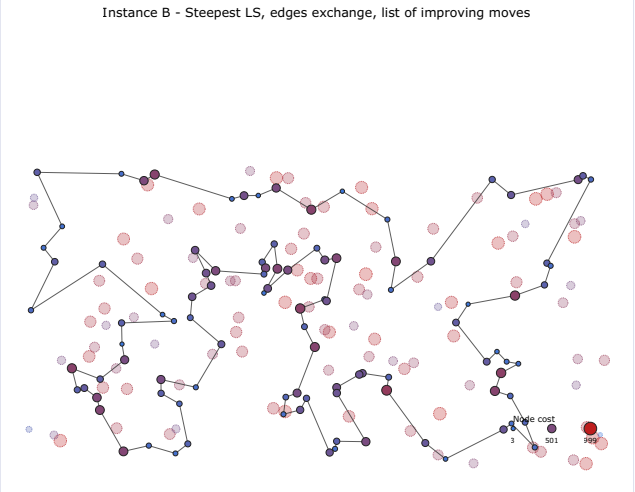


Steepest LS, edges exchange, list of improving moves

Instance B



Steepest LS, edges exchange (baseline)



Steepest LS, edges exchange, list of improving moves

6. Best solutions (node indices, 0-based)

Instance A, Steepest LS, edges exchange (baseline) objective 73561

```
54,9,164,184,186,136,163,29,178,101,147,37,47,1,169,3,50,93,130,159,146,170,75,153,193,78,88,107,64,57,182,138,151,
42,89,106,150,174,176,72,134,84,58,158,192,181,114,98,87,185,35,28,180,141,30,161,187,189,2,171,92,69,127,97,168,11
6,149,63,49,60,31,48,80,108,152,160,83,123,90,132,76,91,157,71,4,51,16,67,144,45,73,135,41,140,53,179,52,177,36,33
```

Instance A, Steepest LS, edges exchange, list of improving moves objective 73561

```
54,9,164,184,186,136,163,29,178,101,147,37,47,1,169,3,50,93,130,159,146,170,75,153,193,78,88,107,64,57,182,138,151,
42,89,106,150,174,176,72,134,84,58,158,192,181,114,98,87,185,35,28,180,141,30,161,187,189,2,171,92,69,127,97,168,11
6,149,63,49,60,31,48,80,108,152,160,83,123,90,132,76,91,157,71,4,51,16,67,144,45,73,135,41,140,53,179,52,177,36,33
```

Instance B, Steepest LS, edges exchange (baseline) objective 50248

```
107,85,51,82,87,179,88,99,171,74,20,14,188,2,110,118,153,90,129,80,127,10,148,114,196,187,169,81,105,94,70,190,104,
54,192,162,76,86,46,72,57,89,156,43,48,96,198,121,147,116,149,28,145,158,125,170,115,180,6,29,132,1,174,62,175,134,
45,60,40,71,155,97,154,140,47,38,21,30,183,69,160,172,136,133,120,8,117,122,151,184,3,27,67,24,34,152,181,167,73,59
```

Instance B, Steepest LS, edges exchange, list of improving moves objective 50230

```
86,46,72,117,160,69,8,120,133,136,172,183,130,30,21,38,47,64,40,155,71,60,45,134,175,62,36,197,29,132,103,6,115,170
,125,140,154,158,65,145,28,149,37,116,147,42,121,198,96,48,156,89,57,185,76,104,190,70,81,94,105,196,114,10,78,54,1
92,80,127,129,90,52,182,153,14,2,20,74,171,99,31,67,24,34,152,181,88,179,87,82,167,51,85,59,73,98,151,122,184,3
```

7. Solution checker

All 4 best solutions above were checked with `checker.py`, which independently re-reads the instance data, verifies that each solution has exactly 100 distinct, in-range node indices, and recomputes length/cost/objective from scratch.

ALL 4 VALID — the recomputed objective matched the value reported by `main.cpp` exactly in every case.

8. Conclusions

- **Same quality, as expected.** The list of improving moves does not change *which* solution steepest LS reaches, only how fast: average 76168.1 vs. 76180.4 on A and 52497.9 vs. 52491.1 on B, with the same number of applied moves (≈133). The small differences come only from moves with equal deltas being taken in a different order. A debug build confirmed this for every run: each applied delta matched the objective recomputed from scratch, and running the baseline from each LM result found no improving move.
- **≈14× fewer move evaluations** (144 k vs. 1.99 M per run). The baseline evaluates the whole neighborhood (≈15 000 moves) in each of ≈133 iterations. LM evaluates it fully only twice: at the start, and once at the end to confirm the local optimum. In between, each iteration evaluates only the ≈800 moves touching the changed edges and nodes. Exactly 2 full evaluations in every run also shows the incremental updates never missed an improving move.
- **≈2.0× faster** (3.84 vs. 7.56 ms on A, 3.73 vs. 7.52 ms on B), much less than the reduction in evaluations. A baseline evaluation is a few nanoseconds in a tight, cache-friendly loop. LM work is more expensive: ≈14 000 insertions into an ordered set per run (most from the initial full evaluation, where ≈11 000 of the ≈20 000 evaluated moves of a random solution, inverted reconnections included, are improving), and browsing past moves that are kept but not applicable (opposite edge directions). Pooling the set's memory cut the LM time from ≈4.9 to ≈3.8 ms; the remaining cost is mostly maintaining the ordered list.
- **Inverted moves and the stopping rule matter.** Without storing the relatively inverted reconnection, a move that becomes applicable after a segment reversal would be missed until the next full evaluation. Keeping them means LM may never become empty, so "stop when LM is empty" does not terminate. The search must stop when nothing is applicable right after a full evaluation.
- The gain grows with the size of the neighborhood: per iteration the baseline costs $O(n^2)$ evaluations, while LM's incremental update costs $O(n)$. For larger instances, combining LM with candidate moves (to shrink the initial and final full evaluations) would be the natural next step.